

MC Dropout Uncertainty and the Qwen3 Orchestrator

What Task 6 delivers, what belongs to the later milestone, where each component runs, and whether a GPU is needed — answered with measurements taken on the development machine.

TASK 6 DUE	RUNS ON	LLM ACCESS	GPU REQUIRED
04 September	Local CPU (VS Code)	OpenRouter API	No

1 First: what Task 6 actually is

The assigned deliverable is one sentence, and it is narrower than the flowcharts:

TASK 6 AS ASSIGNED

Add Monte Carlo dropout at inference to the CNN baseline; report uncertainty estimates and calibration metrics (ECE, Brier score). Feeds the UQ comparison.

There is no LLM in that sentence, no test-time augmentation and no orchestrator. Those belong to the methodology diagrams, which describe the **full pipeline for Milestone 6+**, not this task.

The distinction matters because it protects a dated deliverable. Task 6 can be finished and reported in an afternoon; the orchestrator is a research component with an open novelty question attached to it. Building them together risks delivering neither on time.

TASK 6 — DUE 04 SEP	MILESTONE 6+ — THE ORCHESTRATOR
MC Dropout at inference, T = 50, on the fold checkpoints	Qwen3 reads the uncertainty record and decides what to do
Predictive entropy, aleatoric, epistemic per nodule	Chooses a strategy: TTA / multi-view / alternative specialist
ECE and Brier , deterministic vs MC-averaged	Compares the evidence it gets back and confirms, revises or escalates
Risk-coverage, error-ranking AUROC, reader-agreement correlation	Concept-level explanation of the final decision
Output: <code>uq_interface_contract.json</code> + per-nodule uncertainty	Input: exactly that file

The boundary. Everything on the right consumes Task 6's output through the `uq-1.1` contract; none of it changes Task 6's numbers.

2 Do we need a GPU?

Question

No. Task 6 runs comfortably on the laptop CPU in VS Code.

MC Dropout is **inference only** — no gradients, no optimiser, no backward pass. The GPU was needed for Tasks 4 and 5 because those *train* five fold models each. Task 6 loads those trained checkpoints and samples them.

This is not an estimate. The figures below were measured on the development machine (12 CPU threads, PyTorch 2.10 CPU build) by timing real forward passes at 224×224 and multiplying by the actual workload.

STAGE	RESNET-18	VIT-TINY
One forward pass, per sample	24.7 ms	19.6 ms
Task 6 MC Dropout, full run	11.6 min	9.2 min
Optional TTA on ~96 escalated nodules (8 augs × 50)	15.8 min	12.5 min
5-fold ensemble service, 20-nodule demo	2.1 min	—

Measured single-sample forward cost, then scaled to the Task 6 workload: (322 out-of-fold + 240 validation nodules) × T = 50 passes = 28,100 forward passes.

Both architectures, with TTA, still finish in under an hour. A GPU would make this faster but changes nothing methodologically, and Kaggle's GPU quota is better spent on training.

WHAT YOU DO NEED LOCALLY

The cache (`l1dc_nodules_cache_v2.npz` , ~26 MB) and the **five fold checkpoints** (~215 MB total). Download the `_full.zip` the packaging cell produces. Without the checkpoints there is nothing to sample — this is what was lost last time by not saving a Kaggle Version.

Also set `DATA_SOURCE = "cache"` rather than `"auto"` when running locally. `pydicom` is not installed on this machine, so `"auto"` would try to rebuild, fail, and silently fall back to synthetic phantom data. `"cache"` raises instead.

3 Where each component runs

COMPONENT	WHERE	WHY
Training the fold models (Tasks 4, 5)	Kaggle GPU	Backward passes; already done
MC Dropout sampling	Local CPU	Inference only; 10–12 min measured
ECE / Brier / risk–coverage	Local CPU	NumPy on 322 predictions; seconds
Qwen3 orchestrator	OpenRouter API	Remote inference; no local model weights
TTA / multi-view re-analysis	Local CPU	Inference only; ~16 min measured

Nothing in the orchestrator loop needs a GPU either. Qwen3 runs on OpenRouter’s hardware and is reached over HTTPS, so the only local requirement is an API key and network access.

4 The two methodology diagrams

Comparing them stage by stage: **v2 is v1 with the reasoning detail collapsed**, not a competing design. v1 spells out "compare evidence → confirm / revise / escalate" and adds concept explanation; v2 stops at the final prediction and is explicit about the 2.5D patch and ResNet-18. Same pipeline, two levels of detail.

STAGE	v1	v2
Preprocessing to ROI / 2.5D patch	"Preprocessing → Nodule / ROI"	explicit: ROI → 3 adjacent slices
Initial specialist	CNN / ViT	ResNet-18 or ViT
Prediction + UQ	yes	yes
Qwen3 orchestrator	yes	yes
Strategy choice	TTA / multi-view / alt specialist	same three
Compare evidence, confirm/revise/escalate	explicit stage	folded into "final prediction"
Concept explanation	explicit stage	absent

RECOMMENDATION

Adopt **v2 as the diagram** and keep v1's two extra stages as *named components inside it*. v1's "compare evidence → confirm / revise / escalate" is the orchestrator's second turn, and "concept explanation" is Task 8. Publishing two diagrams invites the question of which one was actually built.

5 How many implementations?

Question — v1×ResNet, v1×ViT, v2×ResNet, v2×ViT?

One implementation, not four.

The architecture is a config flag, exactly as in Tasks 04-01 and 05-01. v1 and v2 are one pipeline. Four near-duplicate notebooks would multiply maintenance and, worse, let the four drift apart — at which point any difference between them stops being attributable to the thing you changed.

What you want instead is an **ablation ladder**: the same code, run with progressively more of the pipeline switched on. That is what demonstrates each component earns its place, and it is what a reviewer will ask for.

0

Deterministic baseline

Single forward pass. Already produced by Tasks 04-01 and 05-01 — no new run needed.

- 1 **+ MC Dropout — this is Task 6**
T = 50 stochastic passes. Reports ECE, Brier, the epistemic/aleatoric split, and risk-coverage against a random-referral control. **The dated deliverable stops here.**
- 2 **+ Threshold routing, no LLM**
Route on the calibrated thresholds alone. This is the control the LLM must beat: if a fixed rule does as well, the orchestrator is adding cost and no information.
- 3 **+ Qwen3 orchestrator**
The LLM reads the uncertainty record and chooses the strategy. Compared against stage 2 on the same nodules.
- 4 **+ Re-analysis actions**
TTA, multi-view and the alternative specialist actually executed, with the evidence fed back for a second decision.

Each rung runs for both architectures via the `MODEL` flag. That is 2 arms × 5 rungs of *configuration*, from one codebase — not four codebases.

STAGE 2 IS THE ONE PEOPLE SKIP

Without it there is no way to say the LLM contributed anything. A fixed threshold on predictive entropy is cheap, deterministic and free; the orchestrator has to beat it on the same out-of-fold nodules before it can be claimed as a contribution.

6 The Qwen3 / OpenRouter layer

OpenRouter exposes Qwen3 through an OpenAI-compatible API, so it is an HTTPS call with an API key — no weights downloaded, no GPU, no local serving. Current listings and prices are on openrouter.ai/qwen.

MODEL	INPUT / OUTPUT PER 1M	COST PER FULL PASS
qwen/qwen3-8b	\$0.117 / \$0.455	≈ \$0.04
qwen/qwen3.7-plus	\$0.32 / \$1.28	≈ \$0.10

Cost for one full pass over 322 nodules, assuming ~400 input and ~150 output tokens per routing decision. Prices as listed on OpenRouter at the time of writing.

Cost is not a constraint — even twenty experimental passes cost a couple of dollars. Start with the smaller model: if a 8B model can make the routing

decision reliably, that is a *better* result for a clinical-deployment argument than needing a frontier model.

Four rules the LLM layer must follow to stay publishable

1. **The LLM never sees the label.** It receives the uncertainty record and nodule metadata only. If ground truth reaches the prompt, every downstream number is void.
2. **Every call is logged verbatim** — model ID, prompt, response, timestamp, and the decision parsed from it. LLM outputs are not reproducible by re-running; the log is the record. Set `temperature = 0` and pin the model version.
3. **The orchestrator's decisions are evaluated out-of-fold too.** A nodule routed by the LLM is still scored by the fold model that never saw its patient.
4. **It decides, it does not predict.** Qwen3 chooses a strategy; the malignancy probability always comes from the imaging model. An LLM asserting a diagnosis from numbers is exactly the hallucination risk the radiology literature documents at 8–15%.

7 Work items, in order

1. **Regenerate the Task 6 notebook against the 04-01 protocol.** The existing `task6_mcdropout_uq.ipynb` was built before the through-plane geometry fix, so it expects the old fingerprint and the old checkpoint naming. It needs rebuilding from the 04-01 cells, as Task 05-01 was.
2. **Run Tasks 04-01 and 05-01 on the rebuilt v2 cache** and download the full bundles. Task 6 cannot start without the checkpoints.
3. **Run Task 6 locally** for both architectures. ~21 minutes total, measured. This completes the dated deliverable.
4. **Add stage 2** (threshold routing, no LLM) as the control.
5. **Then** add the Qwen3 layer, with the logging discipline above.

ONE OPEN QUESTION WORTH SETTLING EARLY

A multi-agent LLM system for lung nodule diagnosis on LIDC-IDRI was accepted at AAAI 2026 (*LungNoduleAgent*), and a separate 2026 framework already performs uncertainty-based clinician-intervention triggering. The orchestrator is therefore not novel on its own.

What still appears unclaimed is using the **epistemic/aleatoric split to choose the kind of escalation** — another model when the model is ignorant, a radiologist when the image is ambiguous — and validating that split against the radiologists' own disagreement. Both are already implemented in Task 6. That argues for presenting Task 6 as the contribution and the orchestrator as its application, rather than the reverse.

